ENGINEERING DESIGN / PART 1 / SECTION B

Enterprise AI, MLOps and Governed MCP Architecture

Principal Engineering Design Document

SWDA ML/AI Engineer Take-Home Assessment | 6 November 2026

DOCUMENT CONTROL	POSITION
Purpose	Part 1B: architectural rationale, controls, lifecycle, operations and implementation gates.
Design basis	Assessment; A1–A4 authoritative Mermaid sources; architecture notes; assumptions register; revision changelog; independent verification report.
Status	Independent-review corrected design narrative; conditional at design level only. No SWDA tenant or production controls tested.
Release scope	Release 1: hourly batch inference. MCP exposure: separate proposed release. Near-real-time serving: future phase.
Protocol target	MCP 2026-07-28; end-to-end Unity Gateway interoperability remains unverified.

Reading convention: F = assessment fact; D = proposed decision; U = unverified dependency; MCP-xx = pending integration test. Product capabilities are cited to official references [Rxx] and never treated as proof of SWDA enablement.

Contents

1 Executive Summary and Scope....3
2 Requirements and Architecture Traceability....4
3 A1: Enterprise AI Architecture....5
4 A2: ML Lifecycle and Reproducibility....7
5 A3: Agent Approval and Governance....9
6 A4: MCP Runtime Architecture....12
7 Monitoring, Observability and AIOps....13
8 Security, Privacy and Responsible AI....15
9 Architecture Decisions and Trade-offs....17
10 Deployment Assumptions and Validation Gates....19
11 Release Strategy and Operations....21
12 Conclusion and References....23
Appendix A Four authoritative diagrams....25

The four figures in Appendix A reproduce the uploaded Part 1A PNG exports without modifying their architecture. The editable Mermaid files remain authoritative for diagram semantics.

Source inventory (source files, not claimed production artefacts): A1–A4 are the four supplied `(3).mmd` files and corresponding supplied PNGs; canonical labels below omit upload suffixes only for stable documentation. Architecture notes: `architecture_notes(3).md`; assumptions: `assumptions_register(2).md`; historical library inputs: `revision_changelog(1).md` and `verification_report(1).md`. Diagrams are not reconstructed, modified or re-rendered from Mermaid here; their earlier rendering checks are historical evidence only. Sources reviewed 6 November 2026.

A1 | canonical label: A1_enterprise_ai_architecture.mmd | supplied: A1_enterprise_ai_architecture(3).mmd | SHA-256: c79e812e58c8f1d91b7c14688ed363ba27dd1e094a503ea772cf33ba55746fde

A2 | canonical label: A2_mlops_lifecycle.mmd | supplied: A2_mlops_lifecycle(3).mmd | SHA-256: e831e6a508f1cd392c2228e4292005fdafd303836d2b7ff6a0cc3b1bba1277c7

A3 | canonical label: A3_agent_registry_approval.mmd | supplied: A3_agent_registry_approval(3).mmd | SHA-256: fe9e36c970719e3fa903e2a8d972470aac70a56874b532699b107e7ec52703f1

A4 | canonical label: A4_mcp_runtime_sequence.mmd | supplied: A4_mcp_runtime_sequence(3).mmd | SHA-256: 885dbcafaaa62d28a7a190375d8d3c00ea298fa34f67e078de9c980ef39d3a44

1. Executive Summary and Scope

SWDA's stated business challenge is to convert a product-team prototype that classifies incoming service requests and recommends relevant programmes or services into an enterprise AI capability that can be operated, assessed and audited. The specified target environment is AWS and Databricks; release one performs **hourly batch inference**, not low-latency interactive scoring. A near-real-time application API and discoverable, approved MCP agents are separately contemplated for later phases. These are assessment facts F01–F05, not assertions about the present SWDA estate. [R01]

The proposed architecture has two distinct execution paths. **The production-intended ML path** ingests controlled source extracts to encrypted S3, validates them through Delta Bronze/Silver/Gold, trains and evaluates versioned models in MLflow/Unity Catalog, and executes a pinned model-and-rules release through hourly Lakeflow Jobs. **The MCP path** exposes a Service Recommendation Agent via an authorised enterprise host, Unity Gateway MCP Service and private MCP backend; its classification and catalogue lookup are deterministic tools, not an invocation of the A1 production ML model. The paths may consume an approved catalogue snapshot but do not share arbitrary data-plane permissions or training privileges. Figure A1 shows this separation.

The engineering thesis is to keep business-policy authority, model lifecycle, and network/tool authorisation separate. Dataset and artefact lineage enable repeatability; approval gates prohibit uncontrolled promotion; least-privilege controls constrain each identity; default-deny checks protect new MCP calls even if Gateway configuration is stale; idempotent batch delivery supports recovery; and telemetry joins technical performance to business outcomes without routinely retaining sensitive request payloads. The design follows AWS Well-Architected ML principles of ownership, protection, reproducibility, resilience and operational optimisation. [R02]

This document addresses the eleven Part 1 criteria and justifies the four existing drawings without changing their component topology. It is deliberately not a build plan for an entire government platform. **Part 1C** separately requires a runnable synthetic, local MCP thin slice, source code, tests and defensible AI-assistance disclosure; none of those executable deliverables is claimed here. The assessment also requires repository-based submission and the candidate's ability to explain and defend AI-generated work. [R01]

Architecture exclusions are deliberate: no online serving in release one; no claim that the MCP classifier is the ML inference model; no unapproved LLM decision-maker; no raw S3/database access for the agent; no self-publishing agent; no assured end-user delegation through Gateway; no assumed approval from a model registry alias; and no invented SLAs, SLOs, RTO/RPO, model accuracy or fairness measurements.

Production boundaries, identity semantics, data contracts and acceptance thresholds remain subject to the U01–U10 validation gates.

2. Requirements and Architecture Traceability

The assessment lists ten system-design concerns plus an eleventh hands-on coding component. Each architecture obligation has a concrete logical location in Figures A1–A4. "Evidence" in the table means a cited design artefact or vendor capability, **not an executed production test**. The design-level source review reports conditional acceptance for A3/A4 and open integration gates. [R01]

Req.	Diagram(s)	Design obligation	Source / rationale	Outstanding validation
01 Data/feature pipelines	A1; A2	S3 → validation/quarantine → Delta Bronze/Silver/Gold; versioned features	F01–F03; D01; A1 source	U07: source contracts, classification, data freshness
02 Experiments/model evaluation	A1; A2	MLflow runs, baseline comparison, candidate checks	D03; A2 staging gates; [R05]	U08: metrics, test set, group reviews
03 Model/prompt/artefacts	A1; A2; A3	UC model versions, MLflow prompt tracking where applicable; immutable agent registry; compatible release manifest	D03–D05; A2/A3	Signing, retention, dependency digest verification
04 CI/CD/CT + gates	A2; A3	CI rejects; CT compares baseline; human/policy gate; CD pinned; agent human approval	A2/A3 source	CI contract and protected-role negative tests
05 Batch/future online	A1; A2	Hourly Lakeflow; clearly separate future API	F03–F04; D02/D13; [R03]	U07/U09; online capacity gate later
06 Monitoring/incidents	A1; A2; A4	Run/quality/drift/cost telemetry, incident/recovery, tool audit	A2/A4 source; [R03,R07]	U09/U10; real alert routing and response drill
07 Drift/feedback/retraining	A1; A2	Outcome feedback; drift proposal back to gated CT	A1/A2 source	U08: label maturity, retrain triggers
08 Security/governance/RAI	A1–A4	IAM/KMS/UC, exact tools, backend PEP, independent approvals, human escalation	D05–D11; [R06,R07,R11]	U03/U04/U06/U10; misuse tests
09 Reliability/recovery/cost	A1; A2; A4	Watermarks, MERGE, run manifest, compatible rollback, spend monitoring, fail closed	D04/D11; [R04]	U09: recovery objective and drill

Req.	Diagram(s)	Design obligation	Source / rationale	Outstanding validation
10 Agent approval/MCP	A1; A3; A4	Ten-state registry, admission/readback, UC selectors, service-principal and backend PEP	D05–D12; [R06,R07,R09]	U01–U06; MCP-01–12 pending
11 Coding thin slice	A1/A3/A4 interfaces	Synthetic local recommend_service with deterministic tools, schemas, tests	F07; [R01]	Separate Part 1C repository execution; NOT covered by design evidence

Figure convention follows a C4-inspired separation of system boundaries and runtime interactions rather than claiming literal C4 levels: A1 is enterprise context/container-like, A2 is a lifecycle process, A3 a governance state/control diagram, and A4 a runtime sequence. The original terminology is retained because diagram-source consistency takes precedence over redrawing notation. [R12]

3. A1: Enterprise AI Architecture

3.1 Source systems, landing and trust boundaries

Figure A1 begins with business systems that own service-request records, programme catalogue information and outcome observations; authorised users consume recommendations and provide feedback. **Approved ingestion connectors** are the only proposed data movement from those systems into **Amazon S3 encrypted landing**. Authentication, service-account scope, KMS keys, private endpoints, source ownership, change-data contract, masking and retention require formal implementation decisions. The data owner remains accountable for source correctness; the platform team controls transport and storage; ML engineers own transformations and model execution. Ingest does not confer model-release permission.

Validation, deduplication and quarantine sit ahead of the analytical layers. **Bronze** preserves the accepted source envelope and ingestion provenance; **Silver** normalises schema, types, identifiers, duplicates and required data-quality constraints; **Gold** publishes versioned feature and catalogue snapshots fit for downstream consumers. Quarantined data must be isolated from business inference, logged with a reason code and reprocessed only through controlled correction. This medallion arrangement is proposed as D01 for repeated ingestion and reproducibility; a direct single-table arrangement would be simpler for a small static dataset but weakens source-level traceability and separation of concerns as inputs grow.

3.2 Model development, approval and hourly inference

Versioned feature engineering reads Gold under Unity Catalog permissions. Candidate training/evaluation writes runs, parameters, metrics, dependency details and artefacts to **MLflow**; governed registered model versions live in **Unity Catalog**.

The registry supplies versioning, discovery, lineage and permissions, but a registry entry does not constitute SWDA production approval. A2 supplies the quality/reproducibility/security gate, followed by a signed or otherwise integrity-protected **approved model + compatible rules manifest**. Databricks documents the MLflow/Unity Catalog integration, but not SWDA's business acceptance process. [R05]

The **Lakeflow Jobs hourly batch** reads the selected Gold inputs and catalogue rules, resolves the exact approved manifest, performs inference, and commits **versioned Delta recommendations with a run manifest**. Its schedule serves F03 and not an arbitrary low-latency target. Each business output includes programme identifiers, evidence references suitable for the consumer, model/catalogue/feature/rules versions, run ID and a review or fallback status as applicable. Delivery to business systems is a separate governed step; retries must not duplicate notifications or case updates. Lakeflow supplies managed task sequencing, schedule/run histories and notifications; SWDA must still implement deduplication, business idempotency and operational escalation. [R03,R04]

3.3 Separate deterministic MCP and future serving

The figure's second downstream consumer is the **restricted read-only catalogue service** fed by an approved Gold snapshot. The private MCP server calls the Service Recommendation Agent, which invokes a **deterministic classifier** and **curated catalogue lookup**. Its rule/reference is explainable, bounded and testable; it is not the trained A1 model and it does not call training pipelines. The agent cannot query arbitrary Delta tables, enterprise databases or raw S3, and the MCP client has no direct backend/data credentials. The same catalogue version may be used across paths, but the output semantics and evaluation baselines remain distinct.

SWDA agent governance is a separate control plane: an **agent approval registry**, **publication/revocation controller**, and **private runtime policy authority** govern the gateway and backend without being mistaken for native Databricks features. The AWS boundary contains S3, approved connectors and a logical IAM/KMS/networking responsibility box, while the Databricks boundary contains managed processing and ML workflows. The thick future-phase connection from an authenticated online application API to a near-real-time endpoint is **not** release-one execution. Any future online deployment requires a fresh capacity, privacy, authorisation, degradation and operations review (D13).

3.4 Consumer-facing service contract

Business application owners should offer a secure user-facing entry point that exposes decision support rather than an unreviewable "AI verdict". Proposed output fields are request/case reference, recommended programme identifiers, current catalogue version, summary of matching conditions, confidence or abstention indicator only if the model supports calibrated interpretation, and a route for human

correction or escalation. Product teams should control entitlements, accessible explanations, corrective feedback, service messaging, and contest/appeal workflows. The backend records provenance and audit IDs while withholding model internals, other users' data and protected attributes unless explicitly authorised. These interface controls are **recommendations for validation**, not facts about an existing SWDA front-end.

4. A2: ML Lifecycle and Reproducibility

4.1 Three separated environments and promotion

Figure A2 gives **development CI**, **staging CT and release evidence**, and **production controlled CD/AIOps** separate responsibilities. A Git commit fixes code, tests, feature/prompt definitions and configuration. CI checks formatting, static analysis, dependencies, vulnerabilities, unit tests, integration/contract tests and protected-secret exposure. Failure rejects the build and requires a new commit. Prompts belong in artefact tracking **where applicable**; the current deterministic MCP classifier does not imply a generative prompt dependency.

Proposed implementation mechanism. Subject to SWDA source-control approval, GitHub Actions is a practical CI orchestrator (an approved enterprise CI runner may substitute). On commit or protected pull request it runs static and dependency checks, unit/contract tests, secret scanning and Databricks bundle validation. A protected merge deploys version-controlled Lakeflow job/pipeline configuration into staging with Databricks Declarative Automation Bundles; staging CT executes on pinned data and logs candidate evidence to MLflow/Unity Catalog. An authorised model-risk/business gate records acceptance, after which a distinct, least-privilege production deployment identity deploys the approved bundle and pins the integrity-protected release manifest. Environment-specific service principals and workload identity federation/OIDC should replace long-lived CI secrets where supported and security-approved; branch/environment controls and a recorded compatible rollback prevent bypass of A2 gates. This is a proposed pipeline, not a configured deployment. [R24,R25]

Passing CI enters reproducible staging. CT trains with declared dataset versions, transformations, feature definitions, seeds, code revision and configuration. MLflow captures experimentation; Unity Catalog stores candidate model versions. The release gate compares candidate against the currently approved baseline on task quality, relevant group-level fairness/coverage, reproducibility, security, operability and cost. Approval must be given by an authorised human **if policy requires it**, or an explicitly recorded preapproved gate policy must authorise promotion. A failing or withheld candidate leaves the production baseline unchanged. The diagram contains no automatic retrain-to-production transition.

Proposed offline evaluation protocol. Use a frozen, representative, labelled holdout and documented temporal/cohort splits, with leakage checks and case-level adjudication. For classification, report macro-F1, per-class precision and recall, class-support counts and a confusion matrix; examine minority-class failure separately rather than relying on overall accuracy. Where programme relevance judgements exist, assess ranking with Precision@K, Recall@K and/or nDCG@K, with K and judgement policy agreed in advance. Measure invalid recommendation rate, catalogue-rule violations, abstention/human-escalation rates and inappropriate coverage. Evaluate operational failures, stale inputs, delayed execution and incomplete outputs against approved business rules. Where lawful, relevant and statistically supportable, stratify error and coverage by reviewed cohorts; report small-sample uncertainty. Compare candidate and approved baseline on identical frozen cases, document regressions and failure slices, and block promotion if any business/model-risk-approved gate fails. No dataset, benchmark result or threshold is asserted. [R26,R27]

4.2 Reproducibility envelope

A reproducible run must capture more than a model name. Its immutable evidence bundle should include **(i)** source commit SHA and dependency lock/container digest; **(ii)** exact dataset/Delta snapshot identifiers and source ingestion watermark; **(iii)** feature code and schema version; **(iv)** model family, parameters and random seeds (with hardware/nondeterminism caveats); **(v)** experiment/run ID and evaluation split or cohort definition; **(vi)** training metrics, error analyses and security/RAI evidence; **(vii)** catalogue/rules version and input/output schema versions; **(viii)** approved model artefact checksum and registry version; **(ix)** environment and authorised sign-off; and **(x)** the canonical release-manifest digest. Repeating a run should recover its lineage even when bitwise model equivalence is not achievable on heterogeneous compute.

The pinned release manifest resolves exact model, code/container, feature schema and transformation, catalogue snapshot, rules, dependencies and configuration versions into a compatible release envelope. A signed (or equivalently integrity-protected) manifest is the production selector, not a mutable `latest` alias. Rollback restores a compatible release and records a new deployment/run ID, never just a model binary. Reproducibility also requires protecting the referenced Delta snapshot against retention/VACUUM expiration for the approved evidence period; retention length and legal basis remain unapproved. A changed prompt, if a later non-deterministic component introduces one, requires a separately tracked version and evaluation. [R04,R05]

4.3 Hourly batching, replay and feedback

Each scheduled run derives a bounded input interval from an explicit event-time watermark and a data-owner-approved lateness policy. Stable source record/version keys support deduplication; late corrections create explicit new versions. Scheduler

job/run IDs identify execution, whereas logical recommendation and delivery keys identify business effects. A proposed Delta `MERGE` uses a unique `(request_id, source_version, approved_release_id)` match key for versioned recommendation rows after upstream deduplication; external delivery also needs a separate durable idempotency/supersession contract agreed with business owners, so a new release cannot accidentally issue duplicate notifications. The run manifest records input snapshot, cutoff, counts, quarantines, output version, checksums and delivery state. MERGE duplicate-source matching rules vary by Databricks Runtime and are not solved merely by using MERGE. [R04,R22]

A restart or backfill must use the recorded input/cutoff and compatible release and perform **idempotent replay**. A technical retry should not send a caseworker another business notification if the original delivery succeeded. Inconsistency triggers quarantine, lineage investigation and replay by an authorised operator. Monitoring distinguishes a missed scheduler trigger, a failed task, a late input window, an empty-but-valid window and failure in downstream delivery. Drift or delayed outcomes generate a **retraining proposal** feeding CT, not an automatic production promotion.

4.4 Containment and rollback

In production, material incident, degradation or integrity concern leads to triage, containment, audit and recovery. Operators may halt delivery, disable a scheduled job, isolate corrupt partitions, or pin the last approved compatible manifest. Restoration and replay should be rehearsed against known source snapshots and output identifiers. Approval for the rollback action should be recorded according to incident policy, while post-incident review updates tests, observability and runbooks. Error budgets, maximum recovery time and replay windows are not defined in the source and must be approved under U09.

5. A3: Agent Approval and Governance

5.1 Authoritative ten-state lifecycle

Figure A3 deliberately places the **SWDA approval registry** outside Unity Gateway. Its ten named states are **DRAFT, REGISTERED, VALIDATED, SECURITY_REVIEWED, APPROVED, PUBLISHED, REJECTED, SUSPENDED, REVOKED, and REAPPROVAL_REQUIRED**. The developer submits an immutable version and digest; the automated validation pipeline checks tool schemas, dependency/security scans, regression/evaluation coverage and negative authorisation; independent security/data reviewers decide whether risk controls suffice; an independent business owner signs off; only a platform-authorised identity attempts publication. A reject does not publish. Emergency suspension or permanent revocation interrupts access; expiry or material change requires renewed review and a **new immutable version** rather than editing an existing approval.

State or transition	Authority / prerequisite	Consequence
DRAFT → REGISTERED	Developer; immutable code/manifest digest; ownership and purpose	No runtime access
REGISTERED → VALIDATED	CI, schema, security, integration and evaluation evidence passes	Candidate eligible for reviews
VALIDATED → SECURITY_REVIEWED	Independent security and data-governance reviewers	Risk evidence attached
SECURITY_REVIEWED → APPROVED	Independent accountable business owner	Exact scope and expiry authorised; not yet public
APPROVED → PUBLISHED	Platform admin/controller checks exact manifest and gateway/backend readback	Only approved client, tool and environment exposed
Any review failure → REJECTED	Validator/reviewer with reason and append-only audit	No publication
PUBLISHED → SUSPENDED	Security/platform emergency authority	Deny new calls while investigating
PUBLISHED/SUSPENDED → REVOKED	Authorised permanent withdrawal	Version remains unusable
PUBLISHED/SUSPENDED → REAPPROVAL_REQUIRED	Expiry, material artefact/policy/tool change, remediation	Previous approval blocked
REAPPROVAL_REQUIRED → DRAFT	Developer submits a new immutable version	Full gate repeats

5.2 Publication control and role separation

The registry record binds `agent_id`, semantic version, owner/purpose, immutable source or container digest, MCP tool names/input-output JSON Schemas, downstream tool allowlist, data classification, approved client principal IDs, IAM/network scope, environment, evaluation/security evidence, independent decisions and timestamps, expiry and linked deployment/config revision. The **SWDA publication controller** checks this exact tuple, verifies an attestable deployed backend digest, and requests authorised Gateway registration or update. A vendor model registry cannot make the SWDA business decision; a Unity Catalog MCP Service cannot be treated as a human approval registry.

The controller must write a non-empty list of exact `include_tool_selectors`, never rely on the documented default (an empty list exposes every tool), and keep automatic inclusion of future tools disabled. It reconciles `EXECUTE` solely to approved host principals plus the necessary parent `USE CATALOG`/`USE SCHEMA` grants; only authorised publishers manage `USE CONNECTION`. Publication is a controlled compare-and-set sequence: verify exact approved registry tuple and backend deployment identity; fetch current MCP Service and its `etag`; apply a conditional API update with the narrow field mask; read back selectors, source connection, grants and backend attestation; record the outcome; only then commit PUBLISHED. If the actual API or delegated tenant permissions cannot perform this safely, publication remains blocked. Configuration drift or conflicting updates suspend exposure rather than widen it. The API documents conditional `etag`

updates and warns that an update mask of `config` replaces omitted optional fields. [R06,R07,R19]

5.3 Runtime policy authority and revocation

The proposed private MCP backend policy enforcement point (PEP) reads an authoritative approval decision for discovery and every newly started `tools/call`. It checks publication state, agent/deployment digest, environment, approved MCP service and tool, authenticated gateway-to-backend identity, policy revision, expiry and revocation; it cannot assume access to the host user identity or Databricks inbound host principal through a shared outbound connection. Host users are authorised by the enterprise host; host service principals are authorised by Unity Gateway. Fail-closed checks reject stale, mismatched, missing or unreachable decisions, and D11 prohibits positive policy caching until verified. Deny-after-revocation-commit for new calls is a target invariant contingent on a linearizable policy read, a defined start-of-call boundary, enforced workload identity and validated clock/concurrency behaviour; it has not been demonstrated. In-flight cancellation and Gateway grant/selector propagation remain separate tests [MCP-07/08/09/12].

Runtime binding contract (proposed): the authoritative decision is keyed by `(agent_id, immutable_version, image_digest, environment, gateway_service_id, backend_service_identity, tool_name, approval_revision)` and returns a signed or authenticated decision, expiry and decision ID. The backend resolves its own deployment identity from a trusted control plane or attested workload configuration, not from a client-supplied field. Any backend rollout changing digest or tool schema is rejected until an approved manifest and publication readback match. The trusted time source, clock-skew allowance, decision-store availability and consistency mode require acceptance testing under U03/U04.

Gateway service policies (`ON CALL`, optional `ON RESULT`) and external guardrail integrations are documented as **Beta** and subject to deployment configuration, attachment and propagation limits. They may add defence-in-depth but are **not** assumed to natively invoke an arbitrary SWDA approval API or provide the only revocation check. Vendor documentation reports a typical 60–90-second propagation period for external policy changes and UI-only attachment; this is incompatible with pretending that administrative Gateway reconfiguration by itself guarantees deny-after-commit. [R07,R08]

Duties must be segregated across developer, ML evaluator, security/data reviewer, business approver, platform publisher, runtime operations and audit reader. No agent or developer can approve itself, alter authoritative state or grant itself `EXECUTE`/`USE CONNECTION`. Approval mutations, publication attempts, rejection reasons, scope changes and denied calls must be logged to an append-only SWDA record with trace correlation but without credentials or routine sensitive payloads.

6. A4: MCP Runtime Architecture

6.1 Participants and credential domains

Figure A4 begins with the **business user** and **enterprise host**. The host authenticates the user and enforces business permissions for a recommendation request. Separately, the host's embedded **MCP client** obtains a dedicated Databricks OAuth **machine-to-machine service-principal** token for calling Unity Gateway. The two authorisation questions are different: may this user ask this business application for assistance, and may this host principal invoke this MCP service? The initial design does **not** represent an unverified end-user identity as if delegated through the Gateway-to-backend connection. A username forwarded in headers, request metadata or tool arguments is not trusted proof of authorisation.

Unity Gateway authenticates the inbound Databricks host service principal, applies `EXECUTE` and parent `USE` grants and filters tools. Its HTTP connection supplies a distinct outbound credential to the private MCP server, which authenticates that gateway-to-backend connection identity before calling the SWDA runtime PEP. Databricks documents that the outbound identity may be a shared provider account or per-user account according to connection mode; neither proves that the original business user or host principal is cryptographically propagated to the backend. In the proposed initial configuration the backend authorises only the approved gateway service, deployed agent and tool scope; user-to-host permission decisions remain the host obligation. The Recommendation Agent uses deterministic classification and restricted read-only catalogue lookup, with no raw S3/database or retraining privileges. [R06,R07,R18]

6.2 Modern protocol and controlled responses

The target MCP `2026-07-28` is a modern stateless protocol revision. It removes protocol-level `initialize` / `notifications/initialized` and session-ID requirements, places per-request version/client capability metadata in `_meta`, and supports optional client-side `server/discover` capability probing. Modern Streamable HTTP requests use the appropriate protocol-version and method routing headers, including `MCP-Protocol-Version`, `Mcp-Method`, and `Mcp-Name` when applicable; the server must reject incompatible versions predictably. The exact header/version/proxy contract must be demonstrated through the actual Unity Gateway registered service, not merely by testing a current SDK against the private server. Pin the SDK to the modern era for this test to prevent a silent 2025-handshake fallback. [R09,R10,R14,R23]

A client may discover a capability and still be rejected at invocation: cached `tools/list` is a presentation aid, never a valid approval token. Gateway denies unselected calls; backend PEP denies unapproved/revoked/expired instances; backend schema checks reject malformed or oversized tool inputs; downstream rule and catalogue checks deny out-of-scope queries. Safe errors should distinguish

policy denial, unsupported protocol, schema rejection, unavailable dependency and internal failure **without leaking secrets or protected records**. The MCP client must parse JSON-RPC and any provider policy envelope, rather than assuming every denial is HTTP 403. Output validation includes bounded result size, authorised fields and catalogue/version provenance.

6.3 Audit, execution limits and failure behaviour

Each invocation should propagate a non-sensitive trace/correlation ID from enterprise host to Gateway and backend; whether the selected Gateway configuration preserves this field is unverified and must be proved in MCP-11. The host logs its user authorisation decision; Gateway separately logs its inbound service principal, MCP Service and tool decision; the backend logs the authenticated outbound connection identity, deployment digest, SWDA policy decision, catalogue version, status and duration. Do not silently merge those into an asserted end-user identity. Databricks documents `system.ai_gateway.usage`, `system.access.audit` and unified tracing; trace capture may include request/response payloads, so trace-table access, content inspection, retention and approved redaction/capture configuration precede sensitive traffic. Routine SWDA logs should contain metadata only, never tokens or raw case contents. [R07,R18,R21]

Requests have schema size limits, tool allowlists, execution deadlines, concurrency/rate ceilings and controlled catalogue query predicates. Dependency failure results in deterministic safe failure or escalation, not broadened access. If the policy service is unreachable, the backend fails closed: availability loss is accepted over unauthorised recommendation execution, while the host presents a controlled service-unavailable status. A later move to delegated end-user identity requires signed-claim validation, principal mapping and replay/impersonation tests under U06.

7. Monitoring, Observability and AIOps

Observability spans ingestion to business outcomes. Platform telemetry explains **whether** a job or call succeeded; ML and business evaluation explain **whether its recommendation remained useful and safe**. AIOps here means instrumented detection, triage correlation and proposed remediation; it is not a claim of autonomous model promotion, self-healing approval or unbounded automated changes. Recommended events must be correlated with run, release, catalogue and trace identifiers while enforcing confidentiality and audit retention policy.

Signal and source	Primary owner / alert route	Operational response / proof
Ingest volume, schema failures, quarantine, duplicates, lateness: connector/S3/Delta quality logs	Data engineering → data owner	Contract breach, hold affected partition; inspect source and replay
Gold feature distribution, nulls, freshness, point-in-time leakage checks	ML/data owners	Compare approved baseline; isolate source change; raise CT proposal

Signal and source	Primary owner / alert route	Operational response / proof
Hourly job trigger, missed run, duration, task failures and backlog: Lakeflow jobs	MLOps/SRE on-call	Retry or contain; run ID audit; overdue window and recovery proof
Batch counts, idempotent MERGE collisions, output delivery ledger, watermark lag	MLOps + business integration owner	Reconcile input/output counts; deduplicated redelivery
Model quality, coverage/abstention, error by approved groups; delayed labels and outcome drift	ML evaluation + business owner	Review sample, label maturity and fairness before gated retraining
Catalogue version and expired programme entries	Programme data steward	Block invalid recommendation candidates; republish approved snapshot
Gateway calls, latency, errors, deny rate, tool selections; UC audit/usage	Platform/Security	Investigate unexpected principal/tool; throttle or suspend
Backend policy freshness, digest mismatches, registry access failures, revocation latency	Security/SRE	Fail closed, incident, policy consistency and deny-after-commit tests
Cost by job/cluster/storage/log volume; abnormal query concurrency	Platform FinOps	Right-size idle schedules, storage/trace retention; review budget
Human escalations, corrections, appeals, recommendation acceptance	Product/business owner	Assess explanation, bias, outcome harm and product guidance

Each metric requires a named data source, owner, dashboard query, alert evaluation window, runbook and escalation path. SWDA must set numerical thresholds only after collecting representative load and jointly approving service expectations. Candidate alerts include sustained batch non-completion, freshness breaches, unexpected drift, unusually high denial rates, configuration drift and silent failure in audit emission; **no threshold is asserted as existing**. Databricks Lakeflow provides built-in run histories/notifications and the Gateway provides usage/audit signals, but SWDA-specific rule/quality metrics and human escalations require application instrumentation. [R03,R07]

Post-deployment quality interpretation. Offline candidate metrics are release evidence, not live quality telemetry. After deployment, log class and recommendation distributions, rule violations, abstention/escalation, invalid outputs, inference failure, watermark lag, late processing and output completeness by pinned release. Join delayed, adjudicated outcomes when available; track feedback lag and sampling bias so early unlabeled cases are not counted as errors or successes. A reviewed change from the approved reference, a catalogue/rule change or accumulating poor outcomes raises investigation and a CT proposal, not immediate self-promotion. [R26]

For incidents, correlate affected cases and output versions, determine blast radius, prevent further delivery or calls, preserve minimal forensic audit evidence, restore approved compatible artefacts, replay idempotently and conduct post-incident improvement. Coverage must include both security incidents (unauthorised tool,

leakage, revoked invocation) and reliability incidents (missed batch, schema drift, stale catalogue, result delivery). Scheduled anomaly detection or automated triage may later be added, but none can bypass approval or independently change production model/agent scope.

Operational acceptance method. The service owner and data owner should first instrument scheduled-start, completion, ingest cutoff, watermark age, delivery receipts and error/retry histories; operations establishes a representative baseline of normal and degraded periods. The business owner then approves batch-completion and freshness objectives, while SRE and security approve availability, error-rate, RTO/RPO and policy-decision consistency/fail-closed expectations through rehearsals and measured load. Model-risk owners approve drift investigation and retraining triggers using evaluated reference cohorts. Finance/platform owners agree job-compute, storage, trace, policy-lookup and replay budgets from attributed metering; all alert thresholds and escalation windows enter the runbook only after sign-off. No numerical SLO, RTO, RPO or budget is presumed.

8. Security, Privacy and Responsible AI

8.1 Control model and governance basis

The proposed default is **least privilege at every hop**: source-system connectors restricted to approved datasets; S3 encryption/KMS and scoped IAM; private routing and secrets management; Unity Catalog grants for Delta and registered models; separate deployment identities; service-level Gateway `EXECUTE` and exact tool selectors; backend inbound service authentication; an independent SWDA policy authority; deterministic downstream tools with read-only catalogue predicates. Approval of a model does not authorise an agent, and registration of an MCP Service does not authorise an underlying container digest. Threat surfaces include source poisoning, data exfiltration, tool expansion, privilege delegation confusion, policy staleness, registry tampering and sensitive logging.

IMDA's **Model AI Governance Framework for Agentic AI** (January 2026; updated May 2026) calls for bounded agent powers, meaningful human accountability, lifecycle controls and end-user transparency. The A3 approval lifecycle, deterministic tools, rejection paths, recorded evidence, privacy minimisation and human escalation are proposed ways to operationalise that guidance. It is **governance guidance, not evidence of statutory compliance**. Applicability of Singapore's PDPA, public-sector data policies, procurement obligations, confidentiality regimes and record-retention rules must be determined by SWDA's authorised legal/privacy and security functions before go-live; this document does not invent regulatory determinations. [R11]

8.2 Threat-control-owner-verification matrix

Threat / failure mode	Proposed preventive and detective controls	Accountable owner	Verification before go-live
Sensitive source leakage / excessive retention	IAM least privilege; S3/KMS; UC access; masking and retention; metadata-only logs	Data steward + security	Access review; encryption tests; sample audit/trace rows
Malicious or malformed source records	Validation/quarantine; typed schema; lineage; deduplication and replay	Data engineering	Poisoning/invalid-schema tests; quarantine integrity
Wrong or biased model outcomes	Approved baseline tests, slice-level review, abstention and appeal	ML evaluation + business risk	Document approved metrics, group data legitimacy and human review
Incompatible model/feature rollback	Pinned signed manifest; versioned dataset/config; compatibility tests	MLOps release owner	Rebuild and rollback rehearsal
Developer/agent self-approval	Separated roles, immutable registry, append-only audit, non-self-publishing admin	Security + business approver	Negative role tests, audit integrity inspection
Unexpected MCP tool expansion	Exact nonempty selectors; auto-include off; gateway grants; backend allowlist	Platform controller	MCP-05, MCP-12 tool/grant drift tests
Unapproved caller or identity spoofing	Host user entitlement; dedicated M2M principal; authenticated gateway ingress; no unsigned delegation	IAM/platform security	MCP-06 and U06 impersonation checks
Revoked agent still executes	Fresh authoritative backend PEP per-call; no positive cache; fail closed	Policy authority + SRE	MCP-07/08 under concurrent load
Private backend bypass	Private route and authenticated gateway service identity; no public ingress	Cloud/platform security	MCP-09 network and direct-call denial
Prompt injection or tool argument abuse	Deterministic classifier, strict schemas, tool scope and output redaction	Application security	MCP-10 and adversarial input tests
Secrets/sensitive values in traces	No tokens; payload capture review; scoped trace tables, retention and deletion	Security/privacy officer	MCP-11, U10 actual trace sampling
Batch duplicate or missed delivery	Watermarks, stable idempotency key, output ledger, missed-run alerts	MLOps/SRE	Backfill, retry and duplicate delivery test

8.3 Transparency and human oversight

At the user interface, recommendations should be clearly characterised as decision support. Display an understandable reason or qualifying criteria, source catalogue version, any known limitation, and a route to human correction or override. Escalate low-confidence/unsupported records, ambiguous programme eligibility, data

conflicts, affected sensitive cases and credible fairness concerns; do not infer approval of a high-impact benefit decision from model confidence alone. Feedback should be reviewed for provenance and representativeness before joining a retraining cohort. Sensitive group evaluation is conditional on lawful/approved collection and access; a “fairness dashboard” without representative, legitimate data would create false assurance.

9. Architecture Decisions and Trade-offs

The selection criterion is **fitness for this assessment’s AWS/Databricks batch-first workload**, not a universal ranking. The table records the preferred logical component, an implementable alternative, why the selected route is justified, the cost/failure consequence, and a concrete switch condition. Choices D01–D13 remain proposed pending environment-level evidence.

Implementation trade-off. For CI orchestration, GitHub Actions is a straightforward option if SWDA permits GitHub; a mandated Jenkins/Azure DevOps platform can fulfil the same checks. Declarative Automation Bundles version job/pipeline configuration across staging and production; Terraform may still be needed for broader workspace provisioning, while unmanaged workspace notebooks risk configuration drift. MLflow/Unity Catalog remain the existing model-evidence and version authority; the SWDA approval registry remains separately authoritative for agents. The chosen deployment mechanism changes how A2 gates are implemented, not the diagram’s CI/CT/CD semantics. [R24,R25]

Decision	Selected rationale	Credible alternative	Cost / failure / switch condition	Basis
Lakeflow Jobs vs Glue/Step Functions/Airflow	Lakeflow keeps Databricks transformations, models, permissions and run lineage in one scheduler.	Glue+Step Functions suits primarily AWS ETL and cross-service choreography; Airflow suits diverse existing DAG operations.	Lakeflow still incurs job compute and workspace coupling; if orchestration is AWS-wide or SWDA already operates Airflow, move the control plane while retaining task contracts.	D02; [R03,R16]
Delta medallion vs flat Parquet/S3	Bronze/Silver/Gold isolates raw/validated/curated states; Delta ACID/upserts suit correction and reproducible snapshots.	Simple append-only Parquet/S3 can suffice for immutable tiny inputs.	More metadata, storage and vacuum/retention governance; choose simpler layers if volume and corrections do not justify them.	D01; [R04]

Decision	Selected rationale	Credible alternative	Cost / failure / switch condition	Basis
MLflow/UC vs SageMaker model registry	Native fit with Databricks training, lineage and governed model versions.	SageMaker registry fits an AWS/SageMaker-centric training and serving estate.	UC/MLflow increases Databricks dependency; switch if platform standards and deployment centre on SageMaker.	D03; [R05,R17]
Pinned manifest vs mutable aliases	Atomic compatibility of model, features, code, rules, catalogue and config.	Aliases simplify operational routing and exploration.	Manifests add signature/promotion machinery; aliases acceptable for discovery, not sole production deployment key.	D04
Unity Gateway vs AgentCore Gateway/direct MCP	UC securable, service grants, managed connection and tool selection match data governance context.	AgentCore Gateway is AWS-native and documents MCP 2026-07-28. Direct MCP reduces a proxy, but places identity, tool controls and audit on SWDA.	AgentCore does not replace Unity Catalog MCP Service grants or Databricks governance. Unity Gateway fits the assumed estate; its 2026-07-28 proxy compatibility, private ingress and revocation remain untested (U01–U04). Prefer AgentCore if verified AWS-native identity, controls, cost and operations justify a switch.	D10/D12; [R06,R07,R15,R20]
SWDA backend PEP vs Gateway-only	Checks approval, digest and expiry on each call independently of Beta policy propagation.	Gateway-only simpler for low-risk tools where platform policy is sufficient.	Adds per-call latency and policy-store dependency; fail closed. Retain until equivalent business authorisation and revocation consistency are proven.	D07/D11; [R08]
M2M host identity vs delegated end-user	Stable approved principal avoids unverified user context propagation.	Delegation can support per-user backend least privilege/audit if signed trustworthy claims propagate.	Host must enforce end-user rights; choose delegation after U06 trust and token mapping tests.	D08; U06
Deterministic tools vs LLM recommendations	Predictable outcomes, bounded permissions, explainable rule references for synthetic MCP scope.	LLM could handle more ambiguous free text and explanations.	Rules have limited semantic coverage; add generative scoring only after new quality, injection, cost and approval assessment.	D09; F07

Decision	Selected rationale	Credible alternative	Cost / failure / switch condition	Basis
Hourly batch vs real-time model API	Matches F03, permits bounded runs, recovery and cost control.	Online serving offers lower request-to-recommendation delay.	Hourly freshness trade-off; adopt online only with approved latency/capacity/availability and privacy cases.	D02/D13; F03/F04

A particularly important commercial distinction is that **platform consolidation is conditional, not proof of lower total cost**. Operating a private PEP and a separate approval service creates additional engineering, but pays for independently enforceable business authority. Conversely, using the existing Databricks estate reduces duplicated lineage infrastructure only if the necessary workspace entitlements, routing, resource quotas and spend governance are established. Cost review must include ingestion, job compute, storage retention, trace logging, M2M token services, policy lookups, operator time and incident replay, not simply model-inference unit cost.

10. Deployment Assumptions and Validation Gates

10.1 Proposed decisions D01–D13

ID	Architectural decision	Acceptance gate
D01	S3 + Delta Bronze/Silver/Gold	Source contracts, UC grants and retention
D02	Lakeflow hourly batch	Tenant scheduling, lateness and replay
D03	MLflow + UC model registry	Catalog permissions, environment isolation
D04	Pinned compatible release manifest	Signing and exact-version reconciliation
D05	Independent SWDA agent registry	Persisted approvals, role separation
D06	SWDA admission/publication controller	API/config readback and negative publish test
D07	Private MCP server + backend PEP	Private ingress, identity and revoke tests
D08	Dedicated host OAuth M2M principal	UC grants and user-to-host entitlements
D09	Restricted read-only catalogue	Classification and unauthorised query negatives
D10	Exact Gateway tool selectors; no auto-add	Real workspace selector/readback tests
D11	No positive policy caching until verified	Fresh consistency, outage and latency tests
D12	Modern MCP 2026-07-28 target	Client→Gateway→server protocol matrix
D13	Online serving is future release	Separate acceptance design

10.2 Unverified U01–U10

ID	Still unverified	Owner / required proof or fallback
U01	Gateway tenant/region MCP feature exists	Databricks admin: create/read/delete sandbox service; otherwise hold publication
U02	Gateway passes MCP 2026-07-28	MCP lead: direct-via-gateway version/header/discovery matrix; explicit legacy contingency or hold
U03	Private route + authenticated backend	Cloud security: no-public-ingress, valid/invalid gateway identities and network tests
U04	Revocation and policy consistency	SRE/security: new-call deny-after-commit, in-flight limits and stale-client tests
U05	Beta external service policies enabled	Workspace admin: preview/access assessment; not baseline prerequisite
U06	Trusted user identity delegation	Security: issuer/audience, signed claims and impersonation tests; M2M baseline otherwise
U07	Source contracts and output SLA	Data owners: schema/classification/freshness/catalogue authority and quality sign-off
U08	Model and fairness acceptance criteria	Business/ML risk owner: approved evaluation and escalation protocol
U09	Recovery targets, on-call, spend	Ops/finance: RTO/RPO/SLO/governance decisions and recovery game day
U10	Trace payload privacy	Security/privacy: inspect sampled real Gateway traces, access/retention and redaction

The **previous Part 1A verification report** records a conditional design pass with two material deployment themes: modern Gateway/protocol interoperability and real revocation/private-network assurance. It reports successful **earlier source/renderer** checks of four diagrams but explicitly **no executed SWDA production integration tests**. Those historical design/render checks are not repeated or elevated to live service evidence in Part 1B.

10.3 Twelve required MCP integration tests

Test	Real integration scenario	Acceptance evidence (all currently NOT EXECUTED)
MCP-01	Direct modern SDK→custom server	Discovery, list and call on 2026-07-28 without legacy handshake
MCP-02	Same client via registered Unity Gateway	Headers, `_meta`, methods, envelopes and errors retained end-to-end

Test	Real integration scenario	Acceptance evidence (all currently NOT EXECUTED)
MCP-03	Malformed/mismatched modern protocol	Reject missing/wrong required version/method metadata; no silent downgrade
MCP-04	Unsupported client revision	Explicit rejection or approved documented older-era adapter
MCP-05	Tool selectors	Unapproved tools hidden/denied; no automatic tool addition
MCP-06	Client identity	Unapproved host denied, tokens absent from logs
MCP-07	Active traffic + revocation/expiry	Every new backend call after commit denied; in-flight behaviour measured
MCP-08	Policy-store outage/staleness	Fail closed; no stale positive authority
MCP-09	Backend/public bypass	Unauthenticated and unapproved direct paths blocked
MCP-10	Invalid schemas/violation	Controlled errors with zero underlying tool invocation or leakage
MCP-11	Trace and audit	Correlated events, no secrets/protected raw payloads in configured stores
MCP-12	Digest/grant/selector drift	Changed artefact or widened config cannot reach PUBLISHED

Release logic: (a) design review checks consistency and authoritative documentation; (b) sandbox integration tests prove particular behaviours; (c) security/privacy/operations owners sign acceptance against defined expectations; only then (d) production change authority may allow release. If MCP-02 fails, the design must not mislabel a 2025-11-25 adapter as native 2026-07-28 support; either defer deployment or design, document and approve an explicit compatibility boundary. No such adapter is presently implemented.

Acceptance criteria to be determined. For release-one operations, the product owner and SRE should sign a service acceptance record specifying the observation window, input cutoff/watermark semantics, source-of-truth dashboard queries, escalation contact and rollback exercise. Evidence must include a representative batch-duration/freshness baseline, deduplicated delivery reconciliation, incident recovery and an itemised cloud spend baseline. The security owner additionally validates new-call denial after an authoritative revocation transaction, policy-store consistency and fail-closed outage behaviour before any governed MCP go-live. U07–U10 remain open until owners approve the measured values.

11. Release Strategy and Operations

11.1 Phased deployment

Release one: batch-first production service. Agree source/data contracts and product handling; implement the S3/Delta workflow; establish MLflow/UC lifecycle

and gated evaluation; approve a compatible manifest; schedule and observe hourly Lakeflow; test idempotent result delivery, rollback and business feedback. The first meaningful go/no-go is driven by data quality, model/rule evaluation, access controls, downstream recovery and on-call readiness, not by an invented headline accuracy or cost figure.

Separate MCP governance release. The local synthetic Part 1C tool may demonstrate SDK quality without AWS or Databricks. A later enterprise MCP rollout requires SWDA registry and independent human approvals, publication controller and readback, private authenticated backend, runtime PEP, approved host principal, exact selectors, real-network proof and MCP-01–12 with particular emphasis on U02–U04. An unavailable Beta policy hook cannot block a properly designed backend PEP, but inability to prove modern Gateway protocol or private ingress must hold or formally re-scope the enterprise rollout.

Future online serving. Only after a new business case and separate architecture approval should SWDA introduce authenticated near-real-time API and model serving. It must establish tail-latency, demand spikes, online/offline feature parity, cache consistency, user authorisation, incident failover, data-residency/privacy and cost guardrails. Figure A1 intentionally sketches only the boundary and not an implemented endpoint.

11.2 Operating responsibility and go/no-go evidence

Responsibility	Proposed accountable group	Go/no-go or BAU evidence
Programme semantics, source contracts, feedback	Business owner / data steward	Approved rules, eligibility definitions, appeal path
Data pipeline and curated catalogue	Data engineering / platform	Quality contract, lineage, watermark and read-only grants
ML evaluation and reproducibility	ML lead / model-risk owner	Baseline comparison, cohort tests, signed evidence
CI/CT/CD and release manifest	MLOps / release manager	Protected release workflow, version/rebuild/rollback proof
IAM, network, backend identity, trace privacy	Security / privacy / Databricks admin	Negative tests, trace inspection, lawful retention decisions
Agent approval and publication	Business approver + independent security reviewer + platform publisher	Different signers, exact digest/grants/selectors and evidence readback
Monitoring, incident and recovery	SRE on-call + service owner	Alerts, runbooks, pager coverage, replay/game day and spend limits
Consumer-facing explanation and escalation	Product/application owner	Accessible explanation, human routing, correction/complaint workflow

A go/no-go record must identify its release scope, decision-maker and linked evidence: release-one batch requires data contract, acceptance baseline, signed compatible manifest, privacy review, successful replay/rollback rehearsal and on-call ownership; separately governed MCP requires verified private ingress, backend identity and digest checks, modern protocol tests, policy consistency, redaction,

audited publication/readback and MCP-01–12; online serving requires its own capacity and authorisation evidence. The authorised owners must approve thresholds and RTO/RPO before production; none are adopted here. Periodic access, drift, model and catalogue reviews, cost and dependency reviews, and recovery exercises are operational proposals, not a pre-existing SWDA cadence.

Owner-approved objectives. Operational sign-off should distinguish measurement from policy: data engineering supplies watermark and completeness history; SRE supplies job/API incidents and recovery drills; the ML evaluator supplies drift/quality analyses; security supplies policy-service denial/consistency evidence; finance supplies cost attribution. Named business, risk and operations owners agree the numerical targets and revisit them after production baseline collection. Until then, monitoring design is documented but service objectives are not approved.

Submission and AI-use declaration. The assessment requests a GitHub repository and disclosure of AI assistance, including representative prompts, obtained responses and how the candidate assessed or applied them. The candidate should document the actual assistance and verification performed; this design document does not claim independent manual review, executed code tests or deployment work not in evidence. [R01]

12. Conclusion and References

The A1–A4 baseline gives a defensible Part 1B design narrative: governed hourly ML inference, reproducible and gated model releases, and a separately controlled deterministic MCP path. The revised document clarifies the host-principal versus backend-connection identity boundary, conditional revocation invariant, conditional MCP API reconciliation and the now-confirmed AgentCore modern-protocol capability without changing the supplied Mermaid topology. A clear, bounded design is preferable to untestable platform breadth. Design-document quality: acceptable with explicit dependencies. Production deployment readiness: not approved. Real tenant protocol, network, identity, policy consistency/revocation, trace privacy, business performance criteria and recovery targets remain unverified.

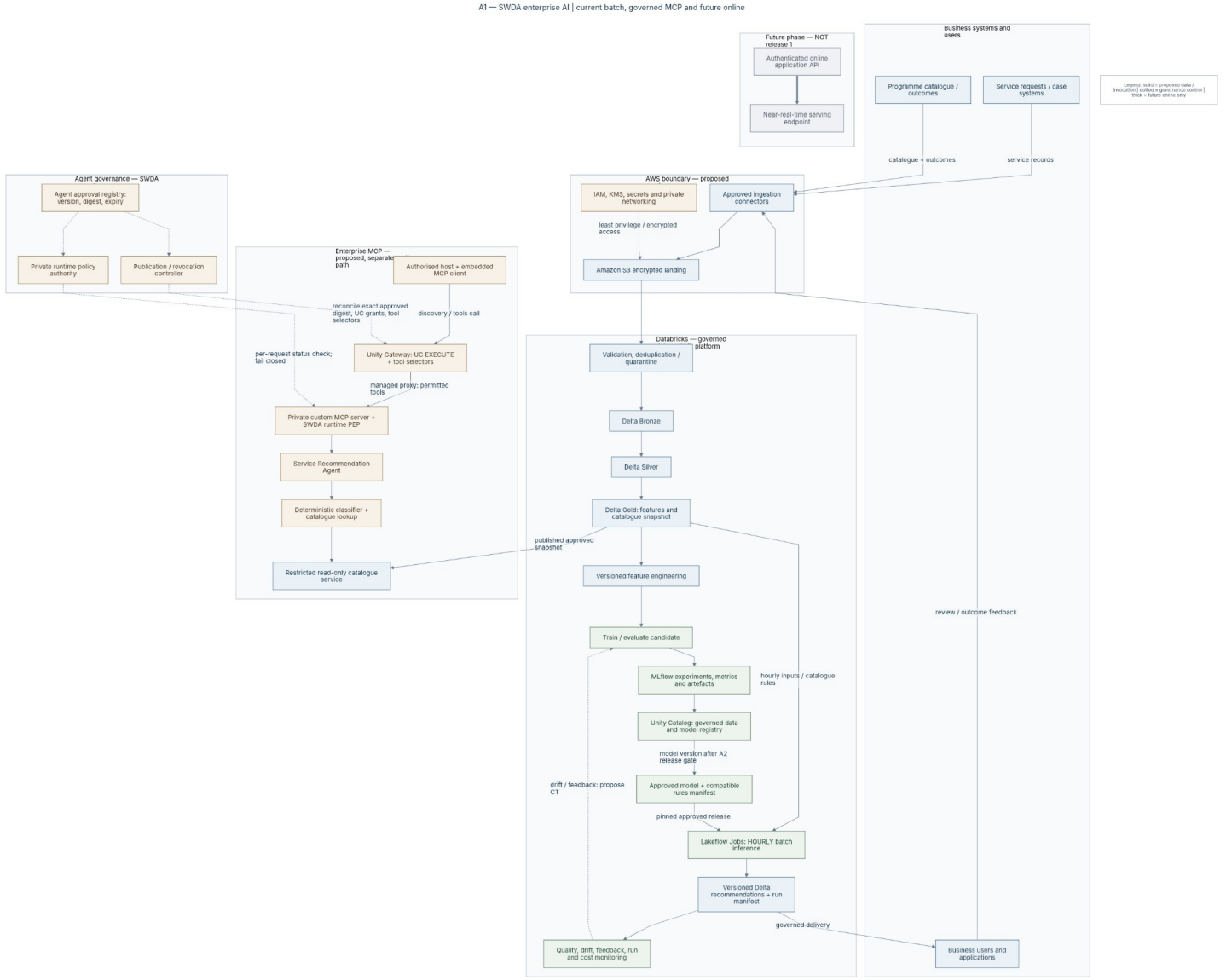
Independent quality-gate conclusion: Part 1B addresses design obligations for requirements 01–10 and traces requirement 11 to the separate Part 1C executable thin slice; traceability is not proof that coding deliverables exist. Four diagrams remain unmodified and authoritative. The earlier Part 1A design/render verification is cited for its limited scope, not repeated as live infrastructure evidence. All twelve MCP integration tests are pending. Any final repository submission must include actual code/test results and a truthful AI-use disclosure independently of this design document.

Primary and authoritative references

[R01] SWDA — ML/AI Engineer Take-Home Assessment, Part 1 and submission instructions.
https://drive.google.com/file/d/1lkvguwI3463kXyKKI-BLXR5WAhbeSIHI/view

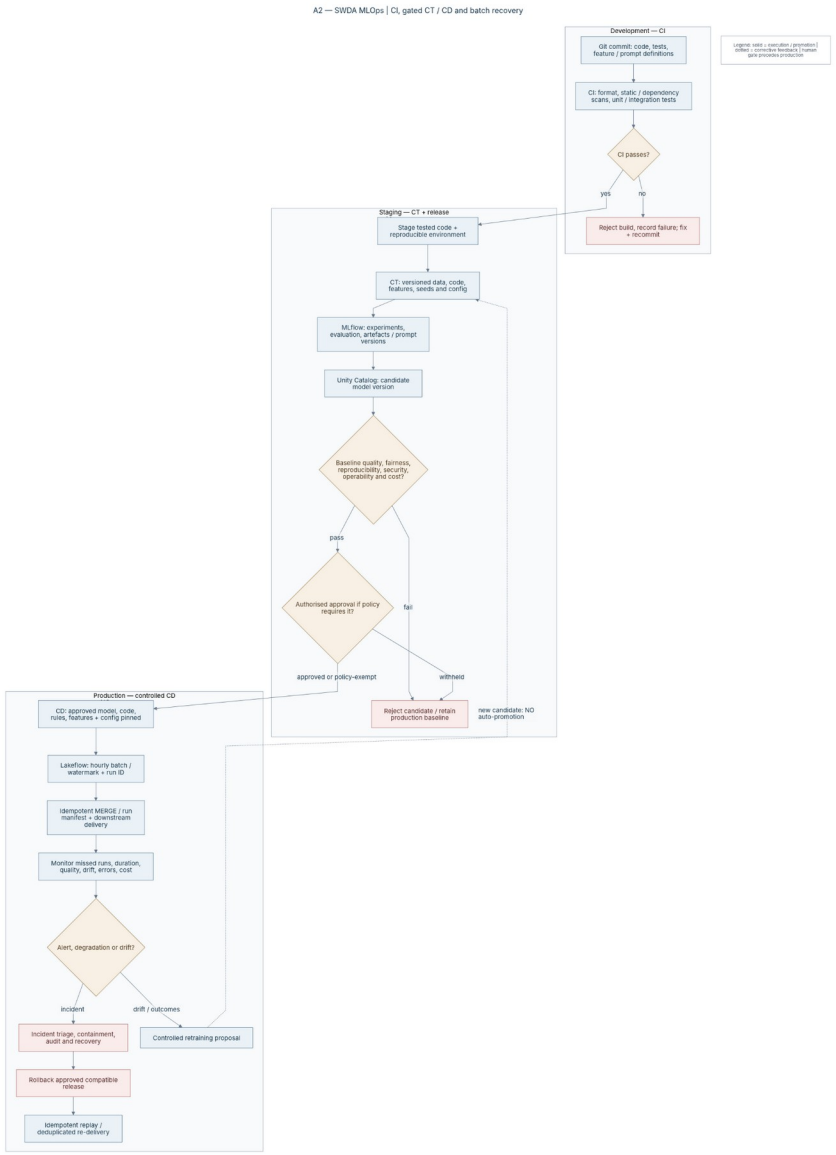
[R02] AWS — Well-Architected Framework: Machine Learning Lens, design principles.
https://docs.aws.amazon.com/wellarchitected/latest/machine-learning-lens/design-principles.html
[R03] Databricks — Lakeflow Jobs, scheduling and orchestration. https://docs.databricks.com/aws/en/jobs
[R04] Databricks — Delta Lake MERGE upsert semantics. https://docs.databricks.com/aws/en/delta/merge
[R05] Databricks — Manage model lifecycle in Unity Catalog; MLflow Model Registry.
https://docs.databricks.com/aws/en/machine-learning/manage-model-lifecycle/
[R06] Databricks — Register an external MCP server / UC MCP Service. https://docs.databricks.com/aws/en/ai-gateway/register-mcp-service
[R07] Databricks — Govern an MCP Service; selectors, Beta policies, usage/audit/tracing.
https://docs.databricks.com/aws/en/ai-gateway/govern-mcp-service
[R08] Databricks — External service policies: Beta limitations, propagation and fail-closed.
https://docs.databricks.com/aws/en/data-governance/unity-catalog/service-policies/external-policies
[R09] MCP project — Specification 2026-07-28: key changes.
https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/changelog.mdx
[R10] MCP project — 2026-07-28 server/discover operation.
https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/server/discover.mdx
[R11] Singapore IMDA — Model AI Governance Framework for Agentic AI (2026, revised May).
https://www.imda.gov.sg/resources/press-releases-factsheets-and-speeches/factsheets/2026/updated-model-ai-governance-framework-for-agentic-ai
[R12] C4 model — Official architecture-visualisation abstractions. https://c4model.com/
[R13] Databricks — MCPs overview and access/networking topics. https://docs.databricks.com/aws/en/agents/mcp-tools
[R14] MCP project — 2026-07-28 release notes. https://blog.modelcontextprotocol.io/posts/2026-07-28/
[R15] AWS — Amazon Bedrock AgentCore Gateway: MCP protocol and use. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway-using.html
[R16] AWS — Step Functions orchestration of Glue jobs.
https://docs.aws.amazon.com/step-functions/latest/dg/connect-glue.html
[R17] AWS — SageMaker Model Registry approval lifecycle.
https://docs.aws.amazon.com/sagemaker/latest/dg/model-registry-approve.html
[R18] Databricks — MCP authentication, inbound user/service principal versus outbound provider credential.
https://docs.databricks.com/aws/en/agents/mcp-tools/authentication-networking
[R19] Databricks — MCP API and SDK reference, conditional etag update and include selectors.
https://docs.databricks.com/aws/en/ai-gateway/mcp-service-reference
[R20] AWS — Amazon Bedrock AgentCore Gateway: documented MCP 2026-07-28 support and configurable supportedVersions (vendor capability; not evidence of SWDA integration).
https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway-using.html
[R21] Databricks — Govern an MCP: usage, audit and unified trace behaviour.
https://docs.databricks.com/aws/en/ai-gateway/govern-mcp-service
[R22] Databricks — Delta MERGE and duplicate source-row matching requirements.
https://docs.databricks.com/aws/en/delta/merge
[R23] MCP TypeScript SDK v2 — Modern era negotiation and explicit revision pinning.
https://ts.sdk.modelcontextprotocol.io/v2/migration/support-2026-07-28
[R24] Databricks — CI/CD on Databricks; Declarative Automation Bundles, GitHub Actions, service principals and OAuth token federation. https://docs.databricks.com/aws/en/dev-tools/ci-cd/
[R25] Databricks — GitHub Actions workload identity federation (issuer/audience/subject and service-principal policy).
https://docs.databricks.com/aws/en/dev-tools/auth/provider-github
[R26] MLflow — Classic ML model evaluation: classifier metrics, confusion matrices, custom business metrics and candidate validation. https://mlflow.org/docs/latest/ml/evaluation
[R27] scikit-learn — Model evaluation and macro-F1/precision/recall with class-support considerations.
https://scikit-learn.org/stable/modules/model_evaluation.html

Appendix A / Figure A1 | Enterprise AI architecture: current hourly batch, governed MCP and future online



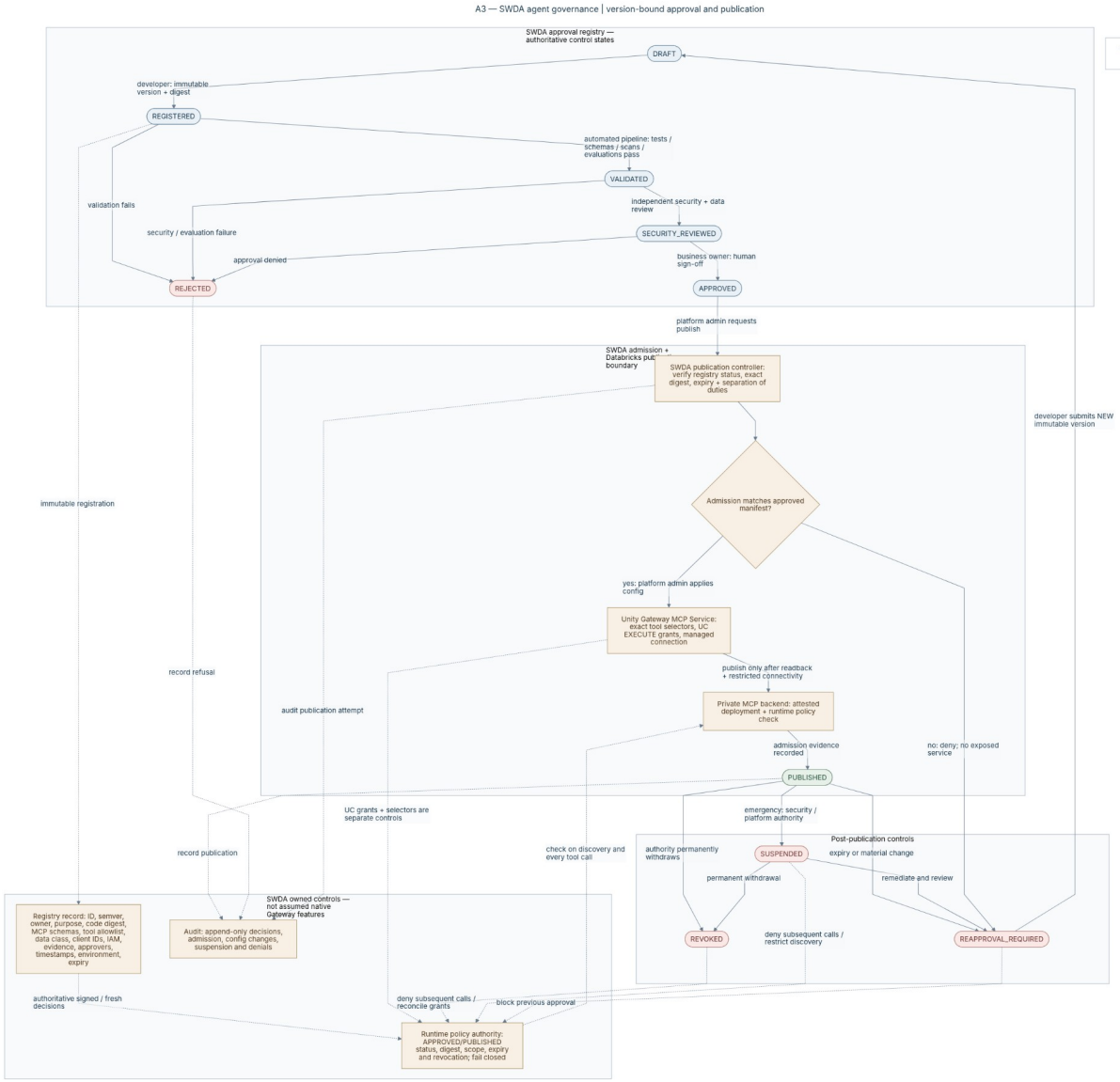
Authoritative A1 source: A1_enterprise_ai_architecture(3).mmd; canonical label and SHA-256 recorded in the source inventory above.

Figure A2 | MLOps lifecycle: CI, gated CT/CD, monitoring and batch recovery



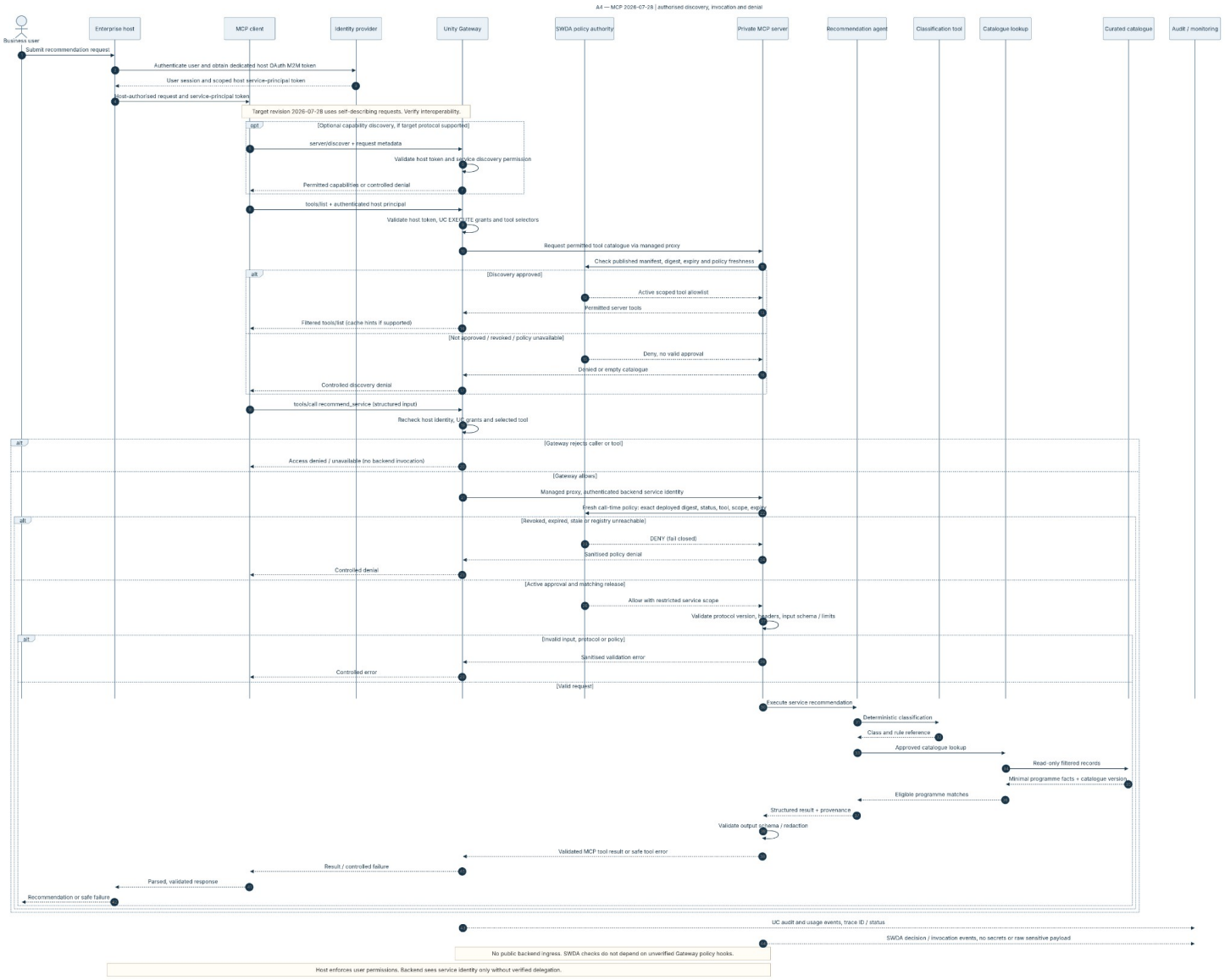
Authoritative A2 source: A2_mlops_lifecycle(3).mmd; canonical label and SHA-256 recorded in the source inventory above.

Figure A3 | Agent governance: ten-state approval and version-bound publication



Authoritative A3 source: A3_agent_registry_approval(3).mmd; canonical label and SHA-256 recorded in the source inventory above.

Figure A4 | MCP runtime: authorised discovery, invocation and denial



Authoritative A4 source: A4_mcp_runtime_sequence(3).mmd; canonical label and SHA-256 recorded in the source inventory above.